



Week 1: Environment Setup & Robotics Foundations

TURTLE Incubator — SO-ARM Software Stack

Program Overview

Our Mission

- Develop the next generation of leaders through hands-on robotics projects, competitions, and community service

Quick Facts

- Founded in 2013 the oldest and largest active robotics organization at TAMU
- 450+ active members across the org
- 18 student-led R&D projects, including SI's own SO-ARM software stack

What is the Software Incubator?



SI in One Line

- A hands-on program that takes complete beginners from zero to programming and controlling a real robotic arm (SO-ARM) in 9 weeks

What SI Really Teaches You

- Not just following tutorials you'll learn to read hardware docs, debug real systems, and think like a robotics software engineer
- Every skill maps directly onto TURTLE's Advanced Projects, where this software runs on real competition robots

Presence & Conduct

- An officer or lab lead must be present whenever the lab is in use
- Disrespectful or unsafe behavior can result in disciplinary action or being asked to leave

Priority & Privileges

- Project tasks always take priority over personal projects for lab tools and space
- Misuse or unsafe practices will result in lab privileges being revoked

When In Doubt

- If you need training or supervision on a tool, ask an officer
- If common sense conflicts with anything here, ask an officer for help

Your Safety Comes First

- You're surrounded by people and resources who can help — if you're ever unsure, ask before you act
- Stop immediately and get clarification if you feel unsafe with a tool

Tools That Require Training

- Soldering iron, 3D printers, drill press, hand and power tools
- Any officer can provide tool-specific training

Required Safety Gear

- Closed-toe shoes, bottoms covering to the lower leg, a sleeved shirt, and safety glasses
- Loose hair and clothing must be secured up and out of the way



Meet the SI Team

Software Incubator Leads & Instructors

Julia Sopala - Development VP



Email: jusopala18@tamu.edu

Discord: juli.a9847



- BMEN 2027
- Meeting Days: TBD
- Experience:
 - OLSN (Prosthetic hand project)
 - R&D Engineering Internship
 - Biomedical Research Lab Experience
 - Hatchling Instructor
 - Former Hatchling
- Fun facts:
 - I want to be fluent in 4 languages
 - I'm 3/5th of the way there
 - I want to learn how to surf

Kaleb - SI Lead



Email: Hkaleb290@tamu.edu

Discord: [thanos0818](#)



- ENGR (2028)
- Meeting Days: TBD
- Experience:
DIRT Software team lead (now project lead)
just a chill dude
- Fun facts:
rides a skateboard

Kayla Sam - SI Instructor



Email: kaysam@tamu.edu

Discord: kkaayyss



- ENGR 2029
- Meeting Days: TBD
- Experience:
 - FASH Electrical Team Lead
 - FTC Robotics
- Fun facts:
 - I have a twin brother
 - I sing in an A&M Choir
 - I'm living in a temporary apartment 🌹



Mohid Ismail - SI Instructor



Email: mohidismail@tamu.edu

Discord: bean.man.



- ELEN 2027
- Meeting Days:
 - the only one ig
- Experience:
 - VIRT
 - Optics Research Lab Experience
 - Former Hatchling
- Fun facts:
 - Have over 7000 hours on LoL
 - Lost a 3v1 to a goat once

Nathan Mersino - SI Instructor



Email:
nathanmersino@tamu.edu
Discord: nathanmersino



- ELEN 2028
- Meeting Days:
 - TBD
- Experience:
 - POBS: Electrical Subteam Lead
 - Former Hatchling
 - Instrumentation Engineering/Transmitter Internship
- Fun facts:
 - In the TAMU Photography Club
 - 1220 Day Duolingo Streak





Icebreakers!

Icebreaker: Name and Fact



How It Works

- Share your name, major, year, and one fun fact about yourself

After Everyone Goes

- Pair up with someone who shared a similar fact and chat for a minute

TURTLE Introductions

- Meet the team leads, learn names, understand how the lab operates

Incubator Goals

- Build the complete software stack of a robotic arm — from zero to autonomous motion

Expectations

- Weekly hands-on deliverables you must demo before leaving each session

Semester Roadmap

- 9 weeks: setup → Python → kinematics → control → estimation → planning → ROS 2

SO-ARM Project Overview

- Every topic this semester maps directly to making the SO-ARM move smarter

Safety & Rules

- Always power down before touching wiring. Never force a joint. Ask before you unplug anything.



Robotics Software Architecture

What Makes a Robot Intelligent?

A robot without software is just a sculpture.

Three things every intelligent robot must do:

- Sense — read the world through cameras, encoders, and IMUs
- Decide — algorithms process that data and choose what action to take
- Act — motors, servos, and grippers execute the decision physically

This sense → decide → act loop runs hundreds of times per second.

The faster and more accurate it runs, the more intelligent the robot appears.

Software vs. Hardware Responsibilities



Hardware does:

- Move joints, resist gravity, handle electrical signals, enforce physical limits

Software does:

- Read sensors, compute what to do, send commands, catch errors, log data

The rule:

- Hardware sets the ceiling. Software determines how close you get to it.
- Bad software on good hardware = broken robot. Good software on bad hardware = still a broken robot.

Your job this semester is the software side — all of it.

Layers of a Robotics Stack



Perception

- Reads raw encoder ticks, camera pixels, IMU data and turns them into useful state: "the arm is at 45°"

Planning

- Given a goal ("pick up the block"), figures out the sequence of positions to move through

Control

- Executes the plan by constantly comparing target vs actual position and correcting the error (PID)

Hardware Interface

- The lowest layer — translates software commands into actual voltage signals to the motor drivers

We will build each layer from scratch, one week at a time.



Development Environment

Installs — What You Need Today



Linux → We use Docker (no need to wipe your machine)

- Install Docker Desktop: docker.com/products/docker-desktop

Git

- Windows: git-scm.com/download/win | Mac: `brew install git` | Linux: `sudo apt install git`

VS Code

- Download at code.visualstudio.com, then install the Dev Containers extension

Verify everything works:

- `docker --version` `git --version` `code --version`

Linux is an open-source operating system used on 96% of the world's web servers and every major robotics platform.

Why robotics runs on Linux:

- ROS 2 (Robot Operating System) is Linux-native
- Real-time control, USB device access, and serial comms are much easier on Linux
- Most robotics libraries are written and tested for Linux first

Key commands you'll use constantly:

- `ls` `cd` `pwd` `mkdir` `rm` `cat` `grep` `chmod` `sudo` `apt install`

Docker packages an entire Linux environment into a portable container that runs identically on any machine.

Image

- A frozen snapshot of an OS + all installed software. Think of it as a USB stick with a full Linux system.

Container

- A running instance of an image. Isolated from your host — you can't break your laptop from inside it.

Commands you'll use:

- `docker compose up -d` # start the container in background
- `docker exec -it so_arm bash` # open a terminal inside the container
- `docker compose down` # stop and remove the container

VS Code is our editor. The Dev Containers extension lets you write and run code directly inside Docker.

Install these extensions now:

- Dev Containers — edit files inside Docker as if they were local
- Python — IntelliSense, linting, and integrated debugger
- GitLens — see who changed what and when, directly in the editor
- ROS — syntax highlighting for launch files and .msg/.srv types

Useful shortcuts:

- Ctrl+` open terminal | Ctrl+Shift+P command palette | Ctrl+Click jump to definition

Git saves a complete history of every change to your code. You can always go back.

The three commands you'll use every session:

- `git add .` # stage all changed files
- `git commit -m "msg"` # save a snapshot with a description
- `git push` # upload your commits to GitHub

Branches let you experiment safely:

- `git checkout -b my-feature` # new branch
- `git checkout main` # switch back to main

Pull Requests let teammates review your code before it merges into main.

SO-ARM Playground

What is the SO-ARM?

The SO-ARM100 is a 6-degree-of-freedom desktop robotic arm built from servo motors and 3D-printed parts.

6 degrees of freedom (DoF) means:

- 6 independent joints — each one rotates to position the arm in 3D space
- More DoF = more flexibility in how you reach a target position

What it can do:

- Pick and place objects, follow trajectories, react to sensor feedback in real time

What powers it:

- Feetech STS3215 servo motors — each has a built-in encoder and PID controller on-chip

Repository Structure



Clone the repo first: `git clone https://github.com/turtle-robotics/so-arm`

- `lerobot/` — core library: configs, datasets, policies, environments
- `common/robots/` — robot hardware wrappers (this is where SO-ARM lives)
- `configs/robot/` — YAML files defining joint names, port, calibration
- `scripts/` — entry-point scripts: teleoperate, train, evaluate
- `tests/` — automated tests for robot and policy code

Everything you touch this semester lives inside `lerobot/common/robots/so100.py` and the configs.

Key Files You Need to Know



lerobot/common/robots/so100.py

- The main robot class. Defines how to connect, read joints, and send commands to the SO-ARM.

lerobot/configs/robot/so100.yaml

- Declares every joint: name, motor ID, USB port, and calibration offsets. Change values here first.

lerobot/scripts/control_robot.py

- The entry point for teleoperation, recording, and replay. You'll run this every session.

lerobot/common/robot_devices/motors/feetech.py

- Low-level serial driver for the Feetech servos. Handles read/write over USB-to-serial.

How the SO-ARM Code is Organized

The codebase has three levels — understand which level you're working at:

Level 1 — Motor Driver (`feetech.py`)

- Speaks directly to servo hardware over serial. Reads raw encoder ticks, writes target positions.

Level 2 — Robot Class (`so100.py`)

- Wraps the motor driver. Converts joint names to motor IDs, handles calibration, reads full arm state.

Level 3 — Control Scripts (`control_robot.py`)

- User-facing. Calls the robot class to teleoperate, record demonstrations, or replay saved motions.

When something breaks, start at Level 3 and trace down — don't start at the driver.

Understanding so100.py — The Robot Class



This file is the heart of everything. Three methods you must understand:

connect()

- Opens the USB serial port, pings each motor to verify it's alive, loads calibration from disk.

read(data_name, motor_names)

- Asks the specified motors for data — position, velocity, or current. Returns a numpy array.
- Example: `robot.read('Present_Position', ['shoulder_pan', 'elbow_flex'])`

send_action(action)

- Sends target positions to all 6 joints simultaneously. Action is a dict of joint_name → angle.

Understanding the Config File

lerobot/configs/robot/so100.yaml controls everything about how the arm is addressed:

- `port: /dev/ttyUSB0`
- The USB serial port the arm is connected to. Change this if your port is different (ls /dev/tty*)
- `motors:`
 - `shoulder_pan: {id: 1, model: sts3215}`
 - `shoulder_lift: {id: 2, model: sts3215}`
 - `elbow_flex: {id: 3, model: sts3215} ... (6 total)`

Each motor ID must match the ID physically set on the servo board (set with Feetech software).

Running Existing Examples — Step by Step



Step 1 Connect the arm

- Plug in USB cable, power on the arm, verify with: `ls /dev/ttyUSB*`

Step 2 Start the container

- `docker compose up -d && docker exec -it so_arm bash`

Step 3 Run teleoperation

- `python lerobot/scripts/control_robot.py teleoperate --robot-path
lerobot/configs/robot/so100.yaml`

Step 4 What you should see

- Terminal prints joint positions at ~30 Hz. Moving the leader arm moves the follower arm in real time.

Step 5 Stop cleanly

- Ctrl+C in terminal. The arm will hold its last position — power off before repositioning by hand.

The robot streams state data continuously while connected. Here's how to read it in Python:

- `from lerobot.common.robots.so100 import So100Robot`
- `robot = So100Robot(config_path='lerobot/configs/robot/so100.yaml')`
- `robot.connect()`

- `# Read all joint positions (returns numpy array, degrees)`
- `positions = robot.read('Present_Position', robot.motor_names)`
- `print(positions) # e.g. [12.3, -45.0, 90.2, 0.0, 15.5, -3.1]`

- `# Read current (load on each motor in mA)`
- `currents = robot.read('Present_Current', robot.motor_names)`

Sending Commands to the Arm

Sending a position command tells each joint where to go. The on-chip PID handles getting there.

- `# Move to a specific pose (all angles in degrees)`
- `action = {`
- `'shoulder_pan': 0.0, # centered`
- `'shoulder_lift': -45.0, # tilted back`
- `'elbow_flex': 90.0, # elbow bent 90°`
- `'wrist_flex': 0.0,`
- `'wrist_roll': 0.0,`
- `'gripper': 20.0, # open`
- `}`
- `robot.send_action(action)`

Modifying Robot Parameters

Three safe places to modify behavior — from safest to most impactful:

1 Config YAML (safest)

- Change joint names, motor IDs, or USB port. No code changes needed.

2 Script arguments (safe)

- `control_robot.py` accepts `--fps` (control rate) and `--warmup-time-s` (startup delay)
- `python lerobot/scripts/control_robot.py teleoperate --fps 10`

3 so100.py directly (careful)

- Modify the `send_action()` method to add speed limiting, smoothing, or safety checks
- This is where real engineering happens — but test in simulation first if possible



Demonstration

Demonstration — Move SO-ARM Using Provided Software



Watch carefully — you will repeat every step of this in the hands-on.

What we'll show:

- 1. Connect arm, verify USB port, launch Docker container
- 2. Run teleoperation script and move the arm by hand on the leader
- 3. Print live joint positions to terminal while arm moves
- 4. Send a hardcoded position command and watch the arm move to it
- 5. Read motor current to see which joints are under load

Demonstration — Exploring Robot States and Commands



After moving the arm, we'll dig into what the robot is actually reporting:

Present_Position

- Each joint's current angle in degrees. This is ground truth for where the arm is.

Present_Velocity

- How fast each joint is moving. Near zero = arm is still. High value = arm is swinging fast.

Present_Current

- Electrical current drawn. High current on a stationary joint = it's fighting gravity or hitting a limit.

Hardware Error Status

- Non-zero means a motor fault. Overload, overheat, or encoder error. Always check this first when something is wrong.



Hands-On

Hands-On — Clone Repository



Goal: get the SO-ARM codebase onto your machine and verify it looks right.

1. Open a terminal (Git Bash on Windows, Terminal on Mac/Linux)

2. Clone the repo

- `git clone https://github.com/huggingface/lerobot`
- `cd so-arm`

3. Explore the structure

- `ls lerobot/` # see top-level folders
- `ls lerobot/common/robots/` # find `so100.py`
- `cat lerobot/configs/robot/so100.yaml` # read the config

4. Verify your USB port (arm must be plugged in and powered)

- `ls /dev/ttyUSB*` (Linux/Mac) | check Device Manager (Windows)

Hands-On — Launch Development Environment



Goal: get the Docker container running and open VS Code inside it.

1. Start the container

- `docker compose up -d`
- `docker ps` # should show so_arm container with status 'Up'

2. Open a shell inside the container

- `docker exec -it so_arm bash`
- You should now see a prompt like: `root@abc123:/workspace#`

3. Verify the install

- `python -c "import lerobot; print(lerobot.__version__)"`

4. Open in VS Code (optional but recommended)

- `Ctrl+Shift+P` → 'Dev Containers: Attach to Running Container' → select so_arm

Hands-On — Run First SO-ARM Program



Goal: make the arm move and read live joint data from Python.

1. Run teleoperation

- `python lerobot/scripts/control_robot.py teleoperate \`
- `--robot-path lerobot/configs/robot/so100.yaml`
- Move the leader arm — the follower should mirror it in real time.

2. Read joint positions in a separate Python script

- `from lerobot.common.robots.so100 import So100Robot`
- `robot = So100Robot('lerobot/configs/robot/so100.yaml')`
- `robot.connect()`
- `print(robot.read('Present_Position', robot.motor_names))`

3. Observe the numbers change as you move joints by hand (power off first for manual moves)



Deliverable

Deliverable — Successfully Run and Modify an Existing SO-ARM Example



You must demonstrate all four of the following to a lab lead before leaving:

1 Repository cloned and structure explored

- Show us so100.py and explain in one sentence what it does

2 Docker container running

- docker ps shows the container as 'Up', you can exec into it

3 Teleoperation working

- control_robot.py runs, arm responds to leader, joint positions print to terminal

4 Parameter modified and explained

- Change one value in so100.yaml or control_robot.py, re-run, and explain what changed and why